理论：RMSNorm

NoteKing | 图文讲义

Author / Channel	MiniMind / Mokio
Duration	4m6s
Generated by	NoteKing — Video to Illustrated Notes
Date	2026-04-03
Video Link	https://www.bilibili.com/video/BV1T2k6BaEeC
GitHub	github.com/bcefghj/noteking
Xiaohongshu	bcefghj

目录

1 课程信息 **2**

2 1. 课程概述与学习目标 **2**

2.1 本节课学习目标 2

3 2. 为什么需要 RMSNorm？ **3**

3.1 2.1 LayerNorm 回顾 3

3.2 2.2 LayerNorm 的计算开销 4

3.3 2.3 RMSNorm 的提出动机 5

4 3. RMSNorm 数学原理 **6**

4.1 3.1 RMS（均方根）定义 6

4.2 3.2 RMSNorm 公式推导 7

4.3 3.3 公式详解 8

4.4 3.4 与 LayerNorm 的对比 8

5 4. RMSNorm 的 PyTorch 实现 **9**

5.1 4.1 基础实现 9

5.2 4.2 详细注释版本 11

5.3 4.3 验证实现正确性 12

6 5. RMSNorm 的计算效率分析 **13**

6.1 5.1 时间复杂度对比 13

6.2 5.2 实际性能收益 13

7 6. RMSNorm 在 Transformer 中的应用 **14**

7.1 6.1 标准位置 14

7.2 6.2 Pre-Norm vs Post-Norm 15

8 7. 关键要点总结 **15**

8.1 本节课要点 16

9 8. 思考题 **16**

10 9. 参考资料 **17**

第 7 集：理论：RMSNorm

理论：RMSNorm 原理与数学推导

1 课程信息

项目	内容
BACKSLASH_PLACEHOLDERtextbf{课程名称}	MiniMind - PyTorch 从零手敲大模型
BACKSLASH_PLACEHOLDERtextbf{课时}	Episode 7/26
BACKSLASH_PLACEHOLDERtextbf{时长}	4 分 6 秒
BACKSLASH_PLACEHOLDERtextbf{主题}	RMSNorm 原理与数学推导

---

2 1. 课程概述与学习目标

本节课我们将深入探讨 **RMSNorm（Root Mean Square Normalization）**的原理与数学推导。RMSNorm 是一种针对深度学习模型优化的归一化技术，在现代大语言模型（如 LLaMA、MiniMind 等）中被广泛采用。

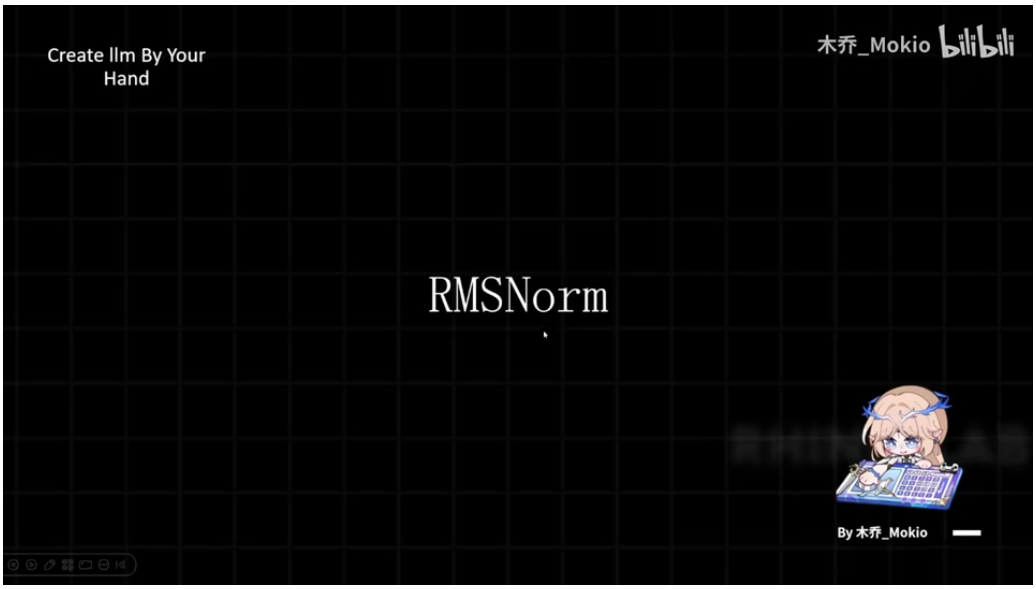

图 1: Video 00:10 [1]

2.1 本节课学习目标

学习背景知识

- 理解归一化在深度学习中的重要性

[1] video timestamp 00:10

- 回顾 Layer Normalization 的基本原理
- 掌握 RMSNorm 的数学推导过程

核心概念

- RMSNorm 的设计动机与核心思想
- RMSNorm 与 LayerNorm 的关键区别
- RMSNorm 在实际模型中的应用

---

3 2. 为什么需要 RMSNorm？

3.1 2.1 LayerNorm 回顾

在正式介绍 RMSNorm 之前，让我们先回顾一下 Layer Normalization 的计算过程。

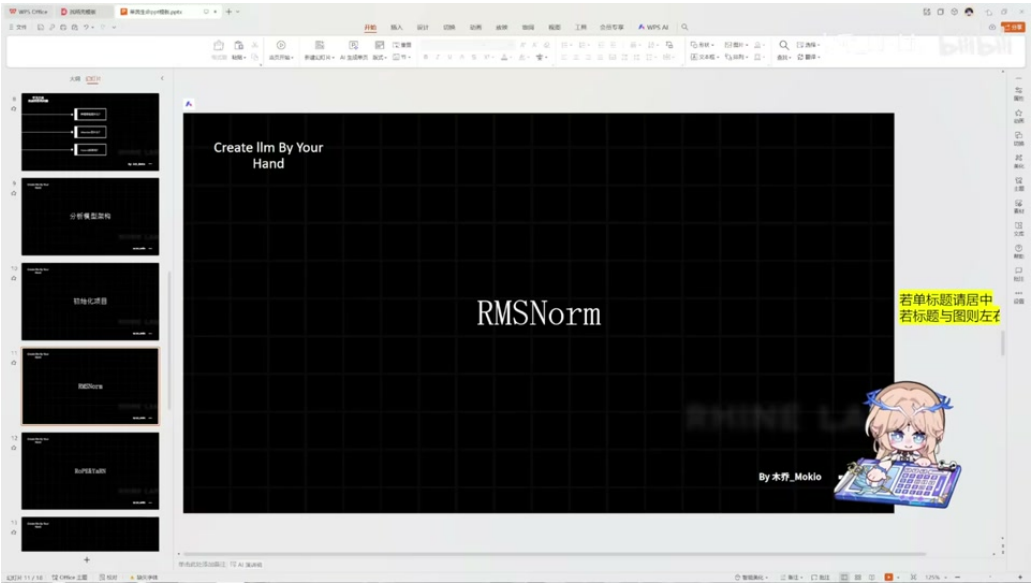

图 2: Video 00:11 [2]

标准的 Layer Normalization 对输入特征进行如下处理：

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

其中：

- $\mu = \frac{1}{H}\sum_{i=1}^{H} x_i$ 是均值

[2] video timestamp 00:11

- $\sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2$ 是方差
- γ, β 是可学习的缩放和偏移参数
- ϵ 是防止除零的小常数

3.2 2.2 LayerNorm 的计算开销

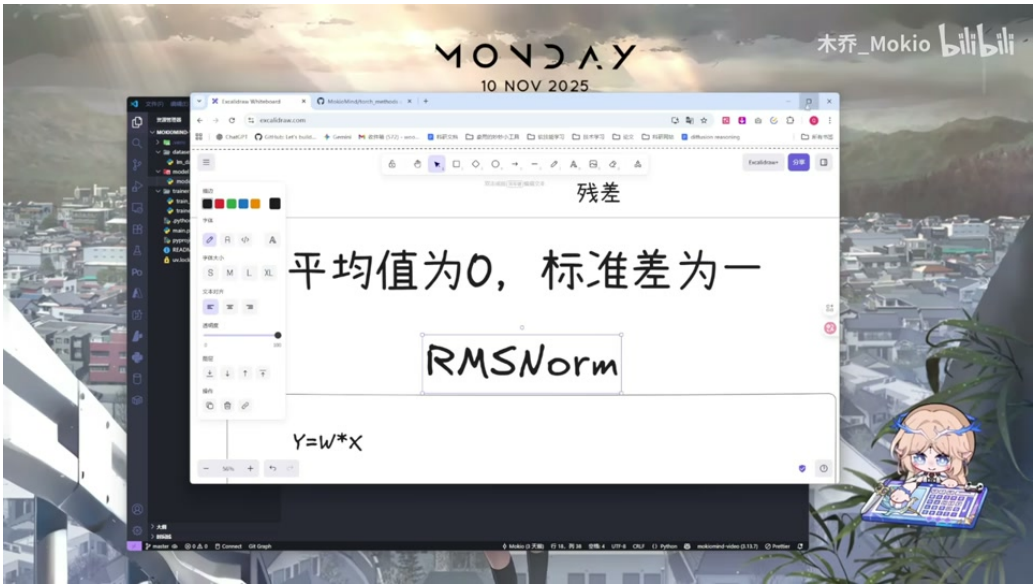

图 3: Video 00:39 [3]

易错点

LayerNorm 需要计算两个统计量：**均值** μ 和 **方差** σ^2，这意味着：

- 需要两遍遍历数据（一遍求和计算均值，一遍求和计算方差）
- 计算图相对复杂
- 在大模型训练时，这些额外计算会累积成显著的时间开销

[3] video timestamp 00:39

3.3 2.3 RMSNorm 的提出动机

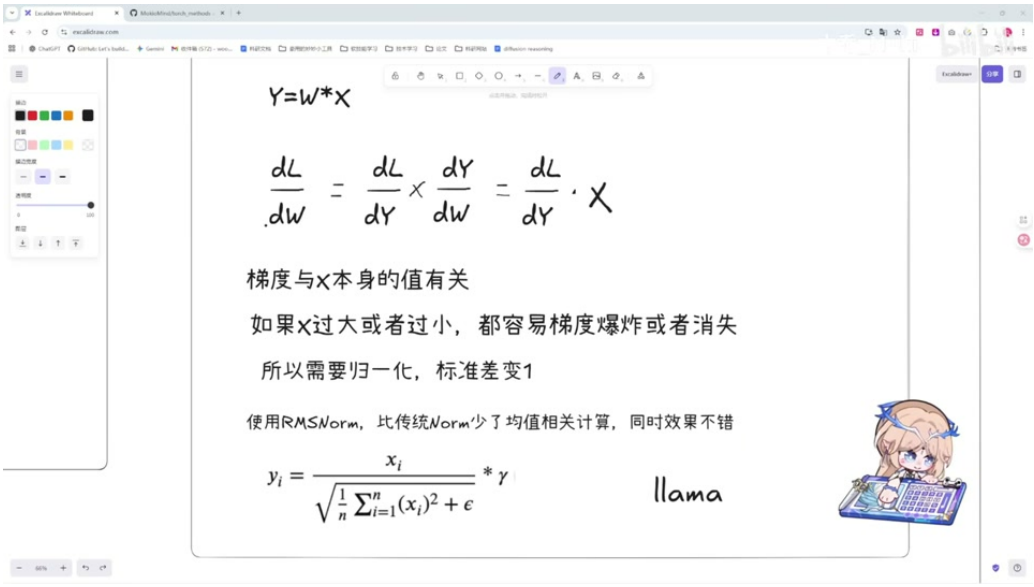

图 4: Video 04:04 [4]

RMSNorm 的核心洞察是：**在许多情况下，均值对模型性能的贡献并不显著。**

背景知识

- 2022 年，Zhang & Sennrich 等人在论文 *"Root Mean Square Layer Normalization"* 中提出 RMSNorm
- 论文链接：https://arxiv.org/abs/1910.07467
- 关键发现：去掉均值中心化步骤后，模型性能几乎不受影响，但计算效率显著提升

[4] video timestamp 04:04

4 3. RMSNorm 数学原理

4.1 3.1 RMS（均方根）定义

图 5: Video 04:05 [5]

RMSNorm 的名称来源于 **Root Mean Square（均方根）**：

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{H}\sum_{i=1}^{H} x_i^2} = \sqrt{\mathbb{E}[x^2]}$$

关键理解

- RMS 衡量的是信号的”幅度”而非”中心位置”
- 与标准差不同，RMS 不需要减去均值
- RMS 可以通过**单遍遍历**计算完成

[5] video timestamp 04:05

4.2 3.2 RMSNorm 公式推导

图 6: Video 00:03 [6]

RMSNorm 的前向传播公式为:

$$y = \frac{x}{\mathrm{RMS}(x)} \cdot \gamma = \frac{x}{\sqrt{\frac{1}{H}\sum_{i=1}^{H} x_i^2 + \epsilon}} \cdot \gamma$$

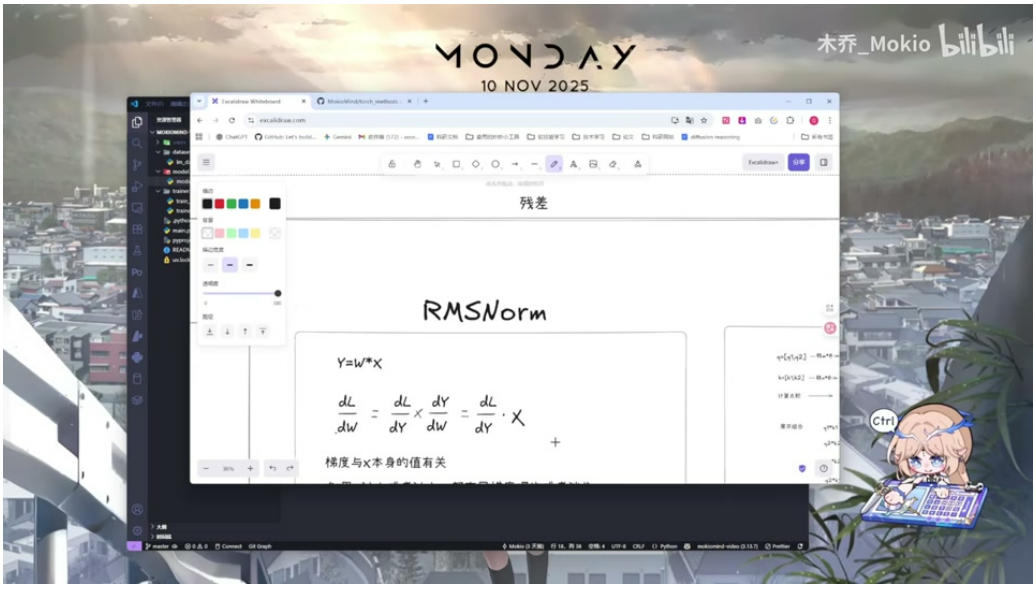

图 7: Video 00:16 [7]

[6] video timestamp 00:03

[7] video timestamp 00:16

4.3 3.3 公式详解

核心公式

$$\boxed{y = \frac{x}{\mathrm{RMS}(x)} \cdot \gamma}$$

其中：

- $x \in \mathbb{R}^{B \times H}$：输入向量
- $\mathrm{RMS}(x) = \sqrt{\frac{1}{H}\sum_{i=1}^{H} x_i^2 + \epsilon}$
- $\gamma \in \mathbb{R}^{H}$：可学习的缩放参数

4.4 3.4 与 LayerNorm 的对比

特性	LayerNorm
BACKSLASH_PLACEHOLDERtextbf{归一化项}	$BACKSLASH_PLACEHOLDERsqrt\{BACK$
BACKSLASH_PLACEHOLDERtextbf{均值项}	需要减去均值 $BACKSLASH_PLACEHOLDE$
BACKSLASH_PLACEHOLDERtextbf{可学习参数}	$BACKSLASH_PLACEHOLDERgamma, BA$
BACKSLASH_PLACEHOLDERtextbf{计算复杂度}	O(2H) 两遍遍历
BACKSLASH_PLACEHOLDERtextbf{计算等价性}	需要计算均值和方差

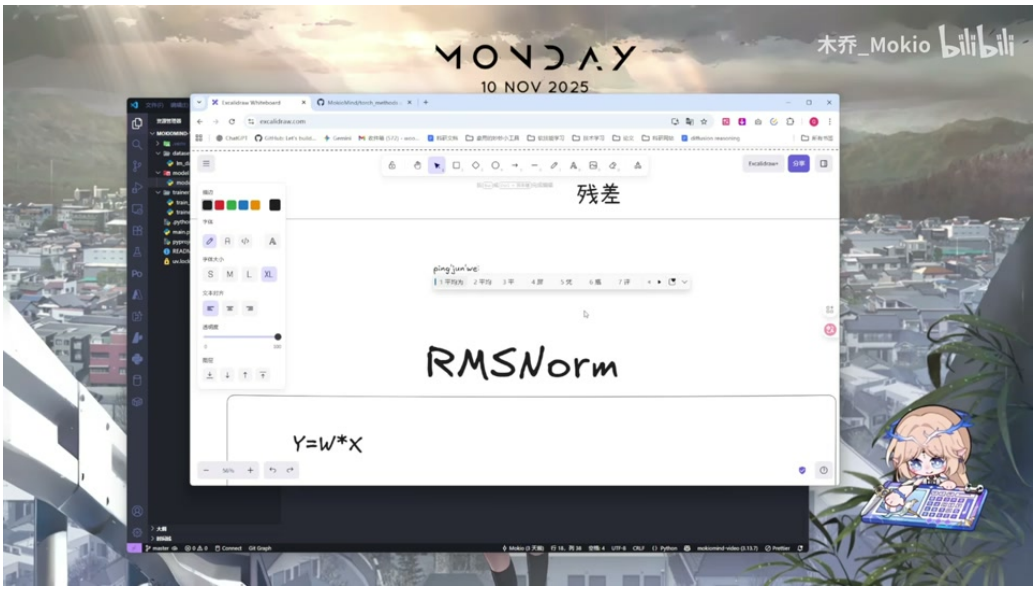

图 8: Video 00:30 [8]

[8] video timestamp 00:30

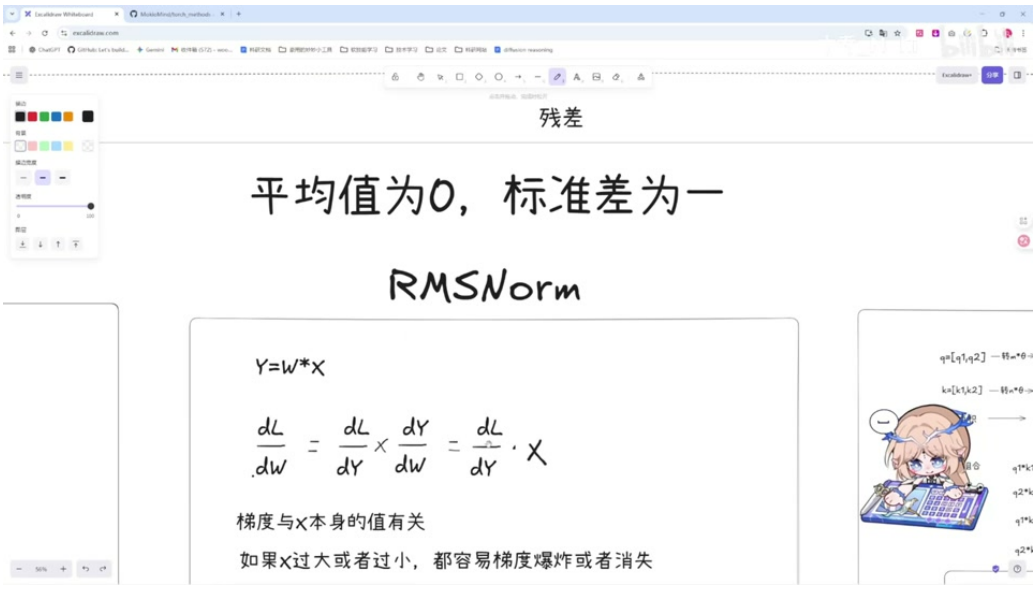

图 9: Video 00:57 [9]

5 4. RMSNorm 的 PyTorch 实现

5.1 4.1 基础实现

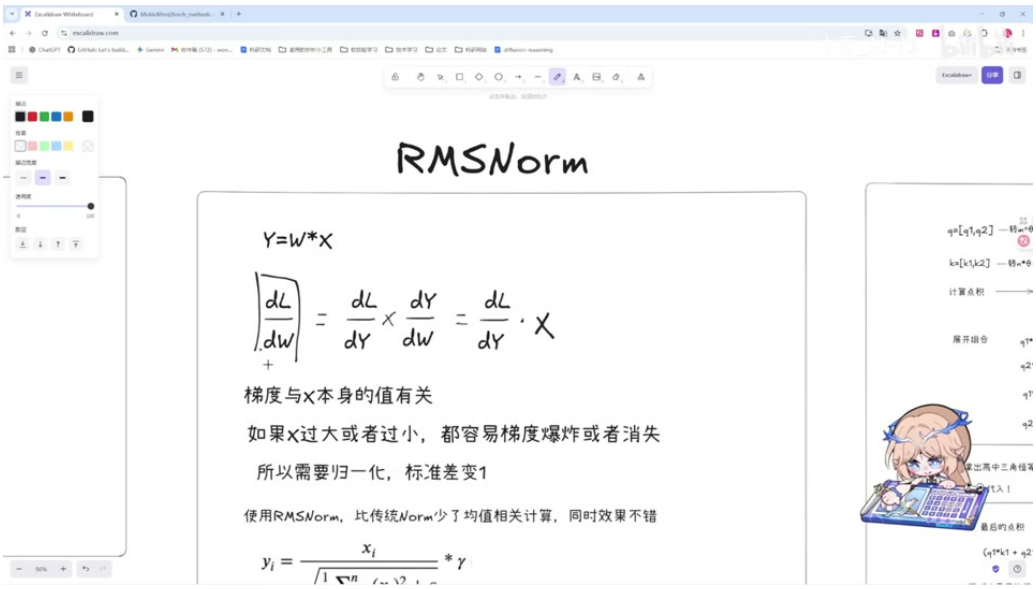

图 10: Video 01:11 [10]

```
import torch
```

[9] video timestamp 00:57

[10] video timestamp 01:11

```python
import torch.nn as nn
import torch.nn.functional as F
import math

class RMSNorm(nn.Module):
    """
    RMSNorm: Root Mean Square Layer Normalization

    论文: "Root Mean Square Layer Normalization" (Zhang & Sennrich, 2019)
    """
    def __init__(self, hidden_size: int, eps: float = 1e-6):
        super().__init__()
        self.hidden_size = hidden_size
        self.eps = eps
        # RMSNorm 只有缩放参数 gamma, 没有偏置参数 beta
        self.weight = nn.Parameter(torch.ones(hidden_size))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: 输入张量, 形状为 (batch_size, seq_len, hidden_size)
               或 (batch_size, hidden_size)
        Returns:
            归一化后的张量
        """
        # 计算 RMS (均方根)
        # x.pow(2) 对每个元素平方
        # mean(-1) 在最后一个维度求均值 (即 H 维度)
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)

        # 归一化并应用缩放
        output = x * rms * self.weight

        return output
```

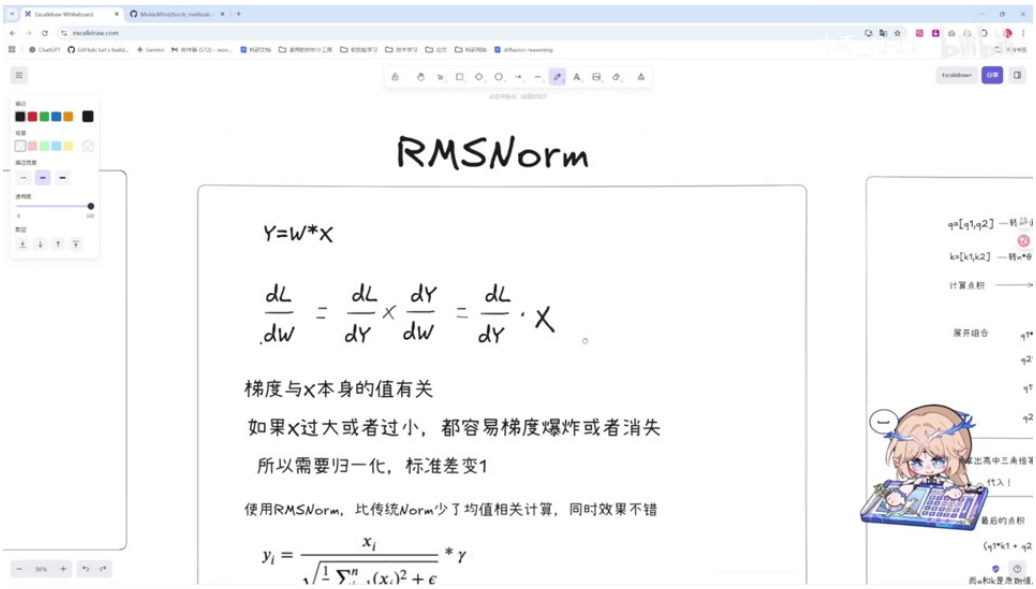

图 11: Video 01:25 [11]

5.2 4.2 详细注释版本

```python
class RMSNormDetailed(nn.Module):
    """
    详细注释的 RMSNorm 实现
    """
    def __init__(self, hidden_size: int, eps: float = 1e-6):
        super().__init__()
        self.hidden_size = hidden_size
        self.eps = eps
        self.gamma = nn.Parameter(torch.ones(hidden_size))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Step 1: 计算每个样本的 RMS
        # 输入形状: (batch_size, seq_len, hidden_size) 或 (batch_size,
            hidden_size)

        # 计算 x^2
        x_squared = x ** 2

        # 在 hidden_size 维度求均值，得到 (batch_size, seq_len) 或 (
            batch_size,)
        mean_of_squares = x_squared.mean(dim=-1, keepdim=True)

        # 计算 RMS = sqrt(mean(x^2) + eps)
        rms = torch.sqrt(mean_of_squares + self.eps)

```

[11] video timestamp 01:25

```
        # Step 2: 归一化
        # x_norm = x / RMS
        normalized = x / rms

        # Step 3: 应用可学习的缩放参数
        # y = x_norm * gamma
        output = normalized * self.gamma

        return output
```

5.3 4.3 验证实现正确性

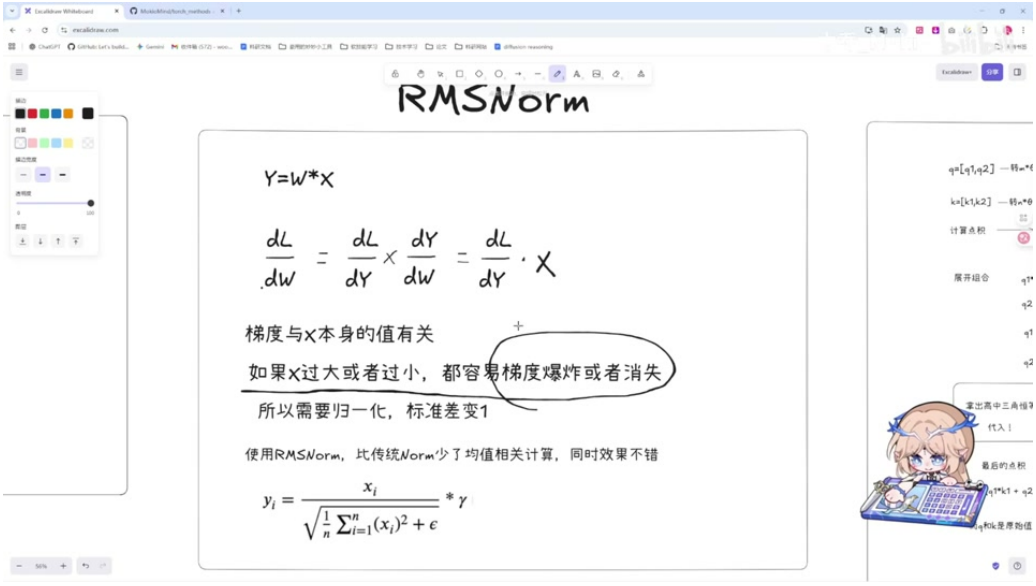

图 12: Video 01:38 [12]

```
def test_rmsnorm():
    """验证 RMSNorm 实现"""
    batch_size = 2
    seq_len = 4
    hidden_size = 8

    # 创建输入
    x = torch.randn(batch_size, seq_len, hidden_size)

    # 初始化 RMSNorm
    rmsnorm = RMSNorm(hidden_size)

    # 前向传播
    y = rmsnorm(x)
```

[12] video timestamp 01:38

```python

    print(f"输入形状: {x.shape}")
    print(f"输出形状: {y.shape}")

    # 验证: 输出的 RMS 应该接近 1 (乘以 gamma 后)
    rms_output = torch.sqrt(y.pow(2).mean(-1))
    print(f"输出的 RMS: {rms_output}")

    # 对比 PyTorch 内置 LayerNorm (不带 bias)
    layernorm = nn.LayerNorm(hidden_size, elementwise_affine=True)
    # 手动设置 LayerNorm 只使用 gamma
    with torch.no_grad():
        layernorm.weight.copy_(torch.ones_like(layernorm.weight))
        layernorm.bias.zero_()

    y_ln = layernorm(x)
    print(f"LayerNorm 输出形状: {y_ln.shape}")

if __name__ == "__main__":
    test_rmsnorm()
```

---

6 5. RMSNorm 的计算效率分析

6.1 5.1 时间复杂度对比

效率对比

对于每个 token 的归一化操作:

操作	LayerNorm	RMSNorm
BACKSLASH_PLACEHOLDERtextbf{减法（计算 x - ）}	需要	不需要
BACKSLASH_PLACEHOLDERtextbf{平方运算}	2 次（用于方差）	1 次
BACKSLASH_PLACEHOLDERtextbf{加法（累加）}	2 次（均值 + 方差）	1 次
BACKSLASH_PLACEHOLDERtextbf{除法}	1 次	1 次
BACKSLASH_PLACEHOLDERtextbf{总体}	约 6H 次基本运算	约 4H 次基本运算

6.2 5.2 实际性能收益

$$Speedup = \frac{Time_{LayerNorm}}{Time_{RMSNorm}} \approx 1.15 \sim 1.30$$

在实际 Transformer 架构中:

- RMSNorm 通常带来 **15% 30%** 的归一化层加速
- 在大模型中，这可能节省 **2% 5%** 的总训练时间

---

7 6. RMSNorm 在 Transformer 中的应用

7.1 6.1 标准位置

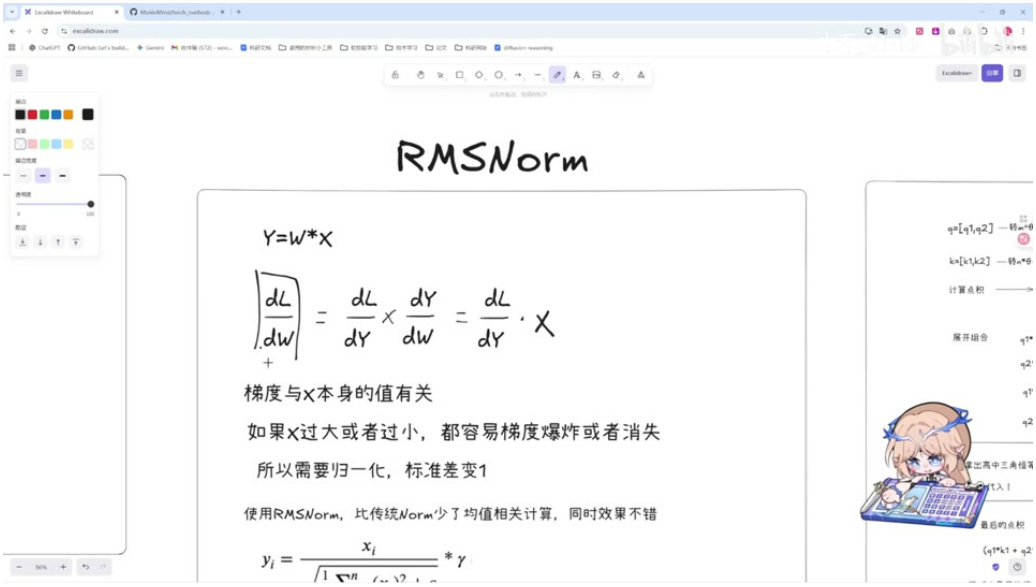

图 13: Video 01:11 [13]

在现代 Transformer 架构中，RMSNorm 通常用于：

```python
class TransformerBlock(nn.Module):
    """Transformer 块示例"""
    def __init__(self, hidden_size, num_heads):
        super().__init__()
        self.attention = MultiHeadAttention(hidden_size, num_heads)
        self.ffn = FeedForwardNetwork(hidden_size)

        # 使用 RMSNorm 而不是 LayerNorm
        self.norm1 = RMSNorm(hidden_size)
        self.norm2 = RMSNorm(hidden_size)

    def forward(self, x, mask=None):
        # Pre-Norm 架构
        x = x + self.attention(self.norm1(x), mask)
        x = x + self.ffn(self.norm2(x))
```

[13] video timestamp 01:11

```
        return x
```

7.2 6.2 Pre-Norm vs Post-Norm

背景知识

- **Pre-Norm**（先归一化）：`x = x + Sublayer(Norm(x))` —更稳定，更常用
- **Post-Norm**（后归一化）：`x = Norm(x + Sublayer(x))` —传统架构

RMSNorm 在 Pre-Norm 架构中表现尤为出色。

---

8 7. 关键要点总结

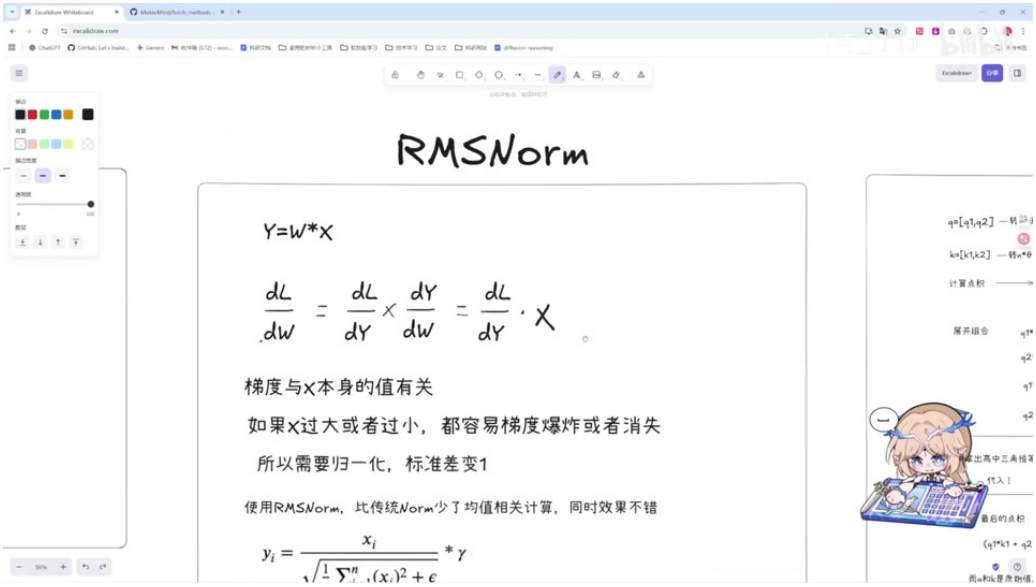

图 14: Video 01:25 [14]

[14] video timestamp 01:25

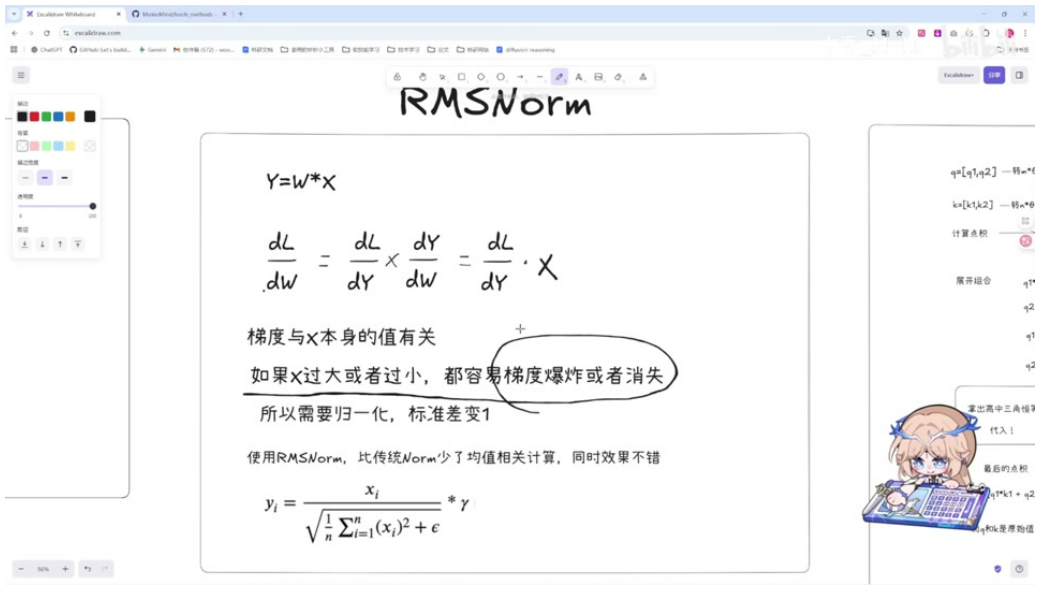

图 15: Video 01:38 [15]

8.1 本节课要点

序号	要点	说明
1	BACKSLASH_PLACEHOLDERtextbf{RMS 定义}	*BACKSLASH_PLACEHOLI*
2	BACKSLASH_PLACEHOLDERtextbf{核心公式}	$y =$ *BACKSLASH_PLACEH*
3	BACKSLASH_PLACEHOLDERtextbf{与 LayerNorm 的区别}	去掉了均值中心化
4	BACKSLASH_PLACEHOLDERtextbf{计算优势}	单遍遍历，减少约 33% 计算量
5	BACKSLASH_PLACEHOLDERtextbf{可学习参数}	仅 *BACKSLASH_PLACEHO*
6	BACKSLASH_PLACEHOLDERtextbf{性能表现}	与 LayerNorm 几乎相当

---

9 8. 思考题

课后思考

思考题 1:

为什么 *RMSNorm* 能够移除均值中心化步骤而不显著影响模型性能？

>

提示：考虑均值在特征分布中的实际作用，以及 *ReLU* 等激活函数对分布的影响。

思考题 2:

[15] video timestamp 01:38

> 如果要在 *RMSNorm* 中添加类似 *LayerNorm* 的偏置参数 β，应该如何修改代码？这会带来什么代价？
>
> 提示：考虑 β 是否真正必要，以及添加后对计算效率的影响。

---

10 9. 参考资料

1. Zhang, B., & Sennrich, R. (2019). *Root Mean Square Layer Normalization.* NeurIPS 2019.

- 论文链接：https://arxiv.org/abs/1910.07467

1. Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization.* arXiv:1607.06450.
2. MiniMind 课程仓库：关注课程 GitHub 获取完整代码实现。

下一节课预告：我们将学习 **注意力机制（Attention Mechanism）**的原理与实现。

讲义编写：*MiniMind* 课程组 最后更新：*2024* 年